

Assignment 2

CNN from Scratch on the MNIST Dataset

Sree Venkata Anjananand Iragavarapu

a1987826

1 October 2026

1. Implementation

The network was built entirely in NumPy. Convolution, pooling, forward and backward propagation and the SGD update are all written by hand. Only the torchvision library was used, to download the MNIST dataset.

1.1 Data preprocessing

Since the pixel values range from 0 to 255, I normalised them by dividing by 255 to bring them into the range 0 to 1. Without normalisation, the first convolution layer would receive inputs 255 times bigger than its weights, making training unstable. Each image was kept as a $1 \times 28 \times 28$ array rather than flattened, since convolution needs to know which pixels are neighbours. The 60,000 training images were split 80/20 into 48,000 for training and 12,000 for validation. The 10,000 test images were left untouched until the final evaluation.

1.2 Architecture

Each 3 by 3 convolution without padding reduces the image size by 2 pixels, while each 2 by 2 pooling layer halves it. Table 1 shows the output shape of each layer.

Layer	Output shape	Parameters
Input	$1 \times 28 \times 28$	0
Conv1, 6 filters 3×3 , ReLU	$6 \times 26 \times 26$	60
MaxPool 2×2	$6 \times 13 \times 13$	0
Conv2, 16 filters 3×3 , ReLU	$16 \times 11 \times 11$	880
MaxPool 2×2	$16 \times 5 \times 5$	0
Flatten	400	0
Dense, ReLU	128	51,328
Dense, softmax	10	1,290
Total		53,558

Table 1: Layers and parameter counts. The second pooling layer drops one row and one column since 11 is odd.

1.3 Forward and backward pass

Convolution. Using four nested loops to perform convolution on 48,000 images would have been much too slow, so I used `im2col`. Each 3 by 3 patch is turned into a row of a matrix, so the convolution becomes just one matrix multiplication. In backpropagation, `col2im` distributes the gradients back to the image. Since patches overlap, their gradients must be added, not replaced.

Output and loss function. The output layer uses softmax with categorical cross-entropy. When the two are combined, the gradient becomes very simple: the difference between the predictions and the labels, divided by the batch size. So I did not have to compute the gradient of softmax separately.

Other layers. ReLU passes the gradient back only through the neurons that were active during the forward pass. Max pooling saves the index of the winning neuron in each window and sends the gradient back to that neuron only.

1.4 Training and model selection

The network was trained with mini-batch SGD using the settings from the brief: batch size 32, learning rate 0.01, 5 epochs, with the data shuffled every epoch. With 48,000 images split into batches of 32, there are 1,500 updates per epoch and 7,500 updates overall.

For model selection, I evaluate the network on the validation set after each epoch and save its weights

whenever validation accuracy improves. The best weights are loaded back before running the test set, so the test accuracy is an honest measure of performance and not a selected number.

2. Results and Analysis

Epoch	Train loss	Train acc	Val loss	Val acc
1	0.4253	0.8735	0.1843	0.9443
2	0.1544	0.9539	0.1326	0.9594
3	0.1156	0.9647	0.1102	0.9682
4	0.0960	0.9703	0.0901	0.9732
5	0.0828	0.9746	0.0838	0.9751

Table 2: Training log. Epoch 5 had the best validation accuracy and was selected.

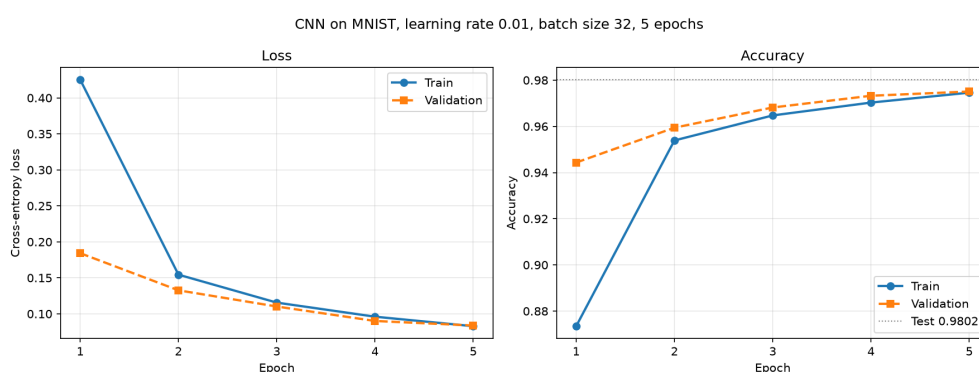


Figure 1: Loss and accuracy over the 5 epochs.

Test accuracy: 0.9802 (9,802 out of 10,000), which exceeds the expected range of 94 to 96% given in the brief.

Observations from the curves.

- Both curves stay close for the entire run, meaning the model is not overfitting.
- Validation accuracy is slightly higher than training accuracy. This is expected, because training accuracy is an average over the epoch while the weights are still improving, whereas validation accuracy is measured once at the end of the epoch.
- Both curves are still climbing at the fifth epoch, meaning the model has not converged yet. It ran out of epochs to learn. I test this in Q2.

Digit	0	1	2	3	4	5	6	7	8	9
Precision	0.984	0.989	0.986	0.986	0.975	0.978	0.994	0.958	0.967	0.986
Recall	0.991	0.993	0.968	0.970	0.986	0.983	0.979	0.986	0.979	0.965
F1	0.987	0.991	0.977	0.978	0.980	0.980	0.986	0.972	0.973	0.975

Table 3: Per-class metrics on the test set.

Error analysis. Performance is even across all classes, with F1 scores ranging from 0.972 to 0.991. Digit 1 is the easiest, because its shape varies very little. The most frequent mistakes are 2 predicted as 7 (18 times), 3 predicted as 5 (11 times) and 9 predicted as 7 (9 times). These digits have similar shapes, so the errors

appear where the classes overlap. On the 198 wrong predictions the model's average confidence was 0.691, compared with 0.980 on the correct ones, so it is much less sure when it is wrong.

3. Challenges and Solutions

For the gradient check before training, I perturbed a sample of weights one at a time by a small amount to see whether the change in loss agrees with the analytically derived gradient.

Challenge 1: the gradient check failed, but not because of a bug. On the first check, the relative error for the convolution biases was 1.0. The reason was the ReLU kink. Most pixels in MNIST are zero, so when a filter passes over an all-zero patch it outputs zero, and with the biases also at zero the pre-activation is exactly zero, where ReLU has no derivative. I counted this directly: out of 129,792 first-layer pre-activations, 80,370 were exactly zero. A small perturbation of the bias pushes these units into positive territory, so the numerical gradient detects a change that the analytical gradient does not. *Solution:* run the gradient check on random inputs with the biases moved slightly off zero (+0.01).

Challenge 2: float32 precision. The check still failed. The network trains in float32 for speed, but the two losses in the central difference differ only in the seventh decimal digit, and float32 stores only about 7 significant digits, so the result was just noise. *Solution:* switch the parameters to float64 only for the check. The worst relative error dropped from 1.0 to 4.21×10^{-8} . Training still runs in float32.

Challenge 3: training time. Loop-based convolution was too slow. Rewriting it with `im2col` and `col2im` reduced every convolution to one matrix multiplication, and all 5 epochs ran in under a minute.

Challenge 4: a silent bug risk in col2im. If `col2im` overwrites overlapping gradients instead of summing them, the loss still decreases, so no error is visible during training. The gradient check is what confirms the operation is correct, which is why I ran it before training.

4. Answers to the Questions

Q1. Why are convolutional layers more effective than fully connected layers for image data?

Far fewer parameters. A dense layer mapping 784 pixels to 128 units requires 100,480 parameters. My first convolution needs just 60, because one small filter is reused at every position (weight sharing). In my model, the two convolutions contain only 940 of the 53,558 parameters in total.

Preserves spatial structure. A flattened image loses the information about which pixels are neighbours, so a dense layer would have to learn this relationship from the training data. A convolution preserves it by design, as each filter only looks at a local patch of the image.

Translation equivariance. If a digit is shifted slightly, its feature map shifts too, so a filter that recognises a stroke detects it regardless of where it appears in the image. A dense layer would treat the shifted image as a brand new input. Pooling also makes the model a little tolerant of such shifts.

Q2. If the model achieves low accuracy, identify two potential reasons and suggest improvements.

Reason 1: underfitting. The model has not learnt enough, either due to a lack of capacity or insufficient training. Sign: training accuracy is low and the training loss is still decreasing at the end. *Improvements:* train for more epochs, adjust the learning rate, increase the number of filters or layers, or use an optimiser like Adam.

This applies to my model, since both curves were still rising at epoch 5. So I trained the same model for 10 epochs instead of 5, without changing anything else:

Epochs	Best epoch	Train acc	Val acc	Test acc
5	5	0.9746	0.9751	0.9802
10	8	0.9835	0.9788	0.9833

Table 4: Same model trained for longer.

Test accuracy increased to 0.9833, showing that the limitation was the training time and not the architecture itself. After epoch 8, validation accuracy stopped improving while training accuracy kept increasing, marking the beginning of overfitting.

Reason 2: overfitting. The model memorises the training data rather than learning general features. Sign: training accuracy keeps rising while validation accuracy stalls or falls, so the gap between them widens. *Improvements:* dropout, weight decay, early stopping at the best validation epoch, a smaller model, or more data (including augmentation).

Q3. Suggest one regularization technique to improve generalization and explain its impact.

Dropout. During training, each neuron is randomly turned off, which forces the network not to rely on any single neuron and to spread what it learns across many. I applied dropout to the 128-neuron layer. I used inverted dropout, which scales up the active neurons during training so nothing needs to change at test time. To demonstrate the effect of dropout, I ran two experiments.

Full data, 5 epochs	Train acc	Val acc	Gap	Test acc
No dropout	0.9746	0.9751	-0.0005	0.9802
Dropout 0.25	0.9642	0.9753	-0.0111	0.9784
5,000 images, 20 epochs				
No dropout	0.9804	0.9495	+0.0309	0.9573
Dropout 0.5	0.9394	0.9537	-0.0143	0.9595

Table 5: Dropout experiments. Gap = train accuracy – validation accuracy.

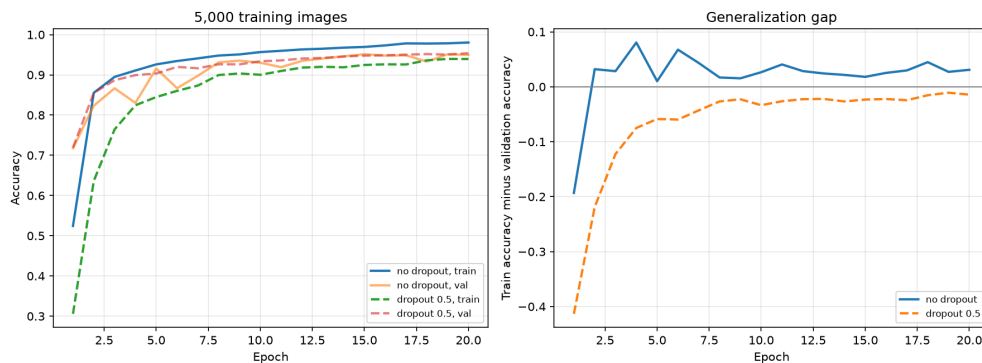


Figure 2: Small-data experiment with and without dropout.

On the full dataset, dropout slightly decreased test accuracy (0.9802 to 0.9784). That is expected, since the model was not overfitting and dropout only removed capacity from a model that was still learning.

Using 5,000 images and 20 epochs, I made the model overfit on purpose. Without dropout, training accuracy reached 0.9804 while validation accuracy stayed at 0.9495, a gap of 3.1 points, and the best epoch was 15. With dropout 0.5, the gap became -1.4 points, the model was still improving at epoch 20, and test accuracy rose from 0.9573 to 0.9595.

Conclusion: dropout improves generalization when the model is overfitting, but when the model is underfitting it reduces accuracy.

Statement on the Use of Generative AI

I used both Gemini and Claude to conduct research and develop the outline of this project. Furthermore, I used these two AI tools to help me understand the math behind backpropagation and numerical gradient checking. They helped me in polishing the code and providing language editing, feedback according to the rubric of evaluation and also in Latex formatting and typesetting of the report. All final content and calculations are my own.